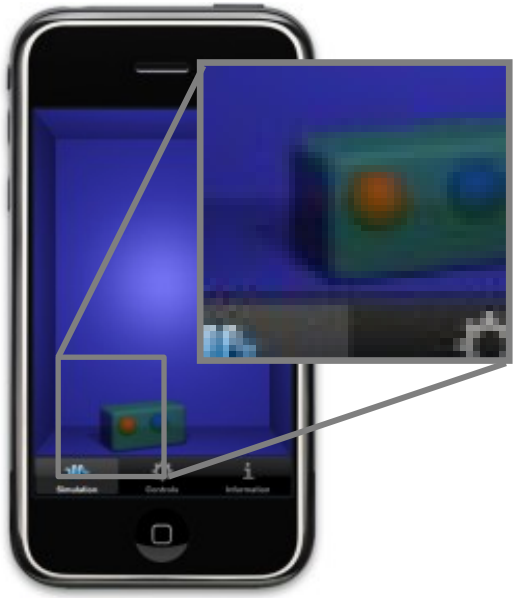
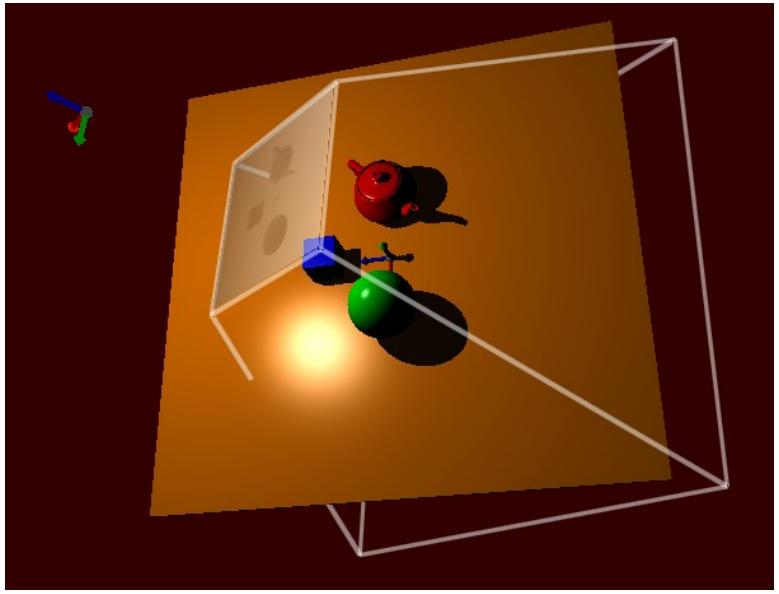
Shadows and Object Order Rendering

Cheap Shadows, Shadow Maps, Shadow Volumes





- **Cheap shadows:** projecting geometry onto planar surfaces and draw as a shadow
- **Shadow maps**: texture of depth drawn from the light view and then used to compare light distance when drawing from camera
- **Shadow volume**: pixel accurate sharp shadows (not covered in tis course)

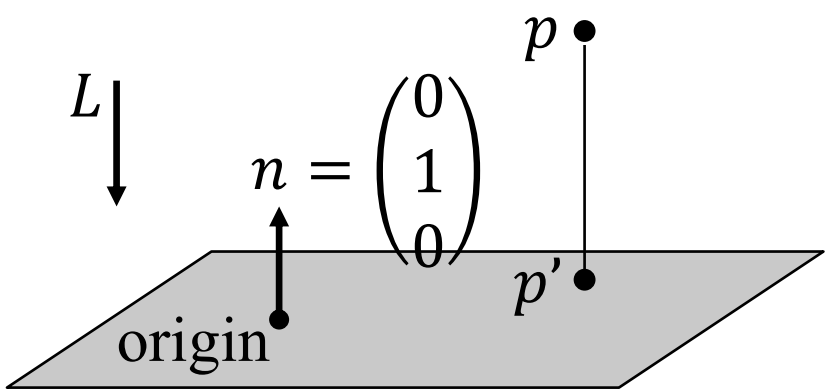
CHEAP SHADOWS

Cheap Shadow Projection

- Redraw geometry projected onto floor plane using a dark solid colour (i.e., *draw the shadows*)
  - Only good for planar shadows in simple environments
  - No self shadowing, but often very compelling

Cheapest Shadow Projection

- Assume directional light from above, (0,-1,0)
- Assume plane with y-axis normal containing the origin
- Projection simply sets the y value of points to zero

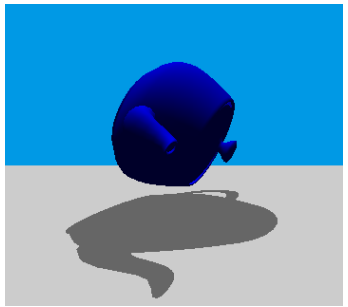


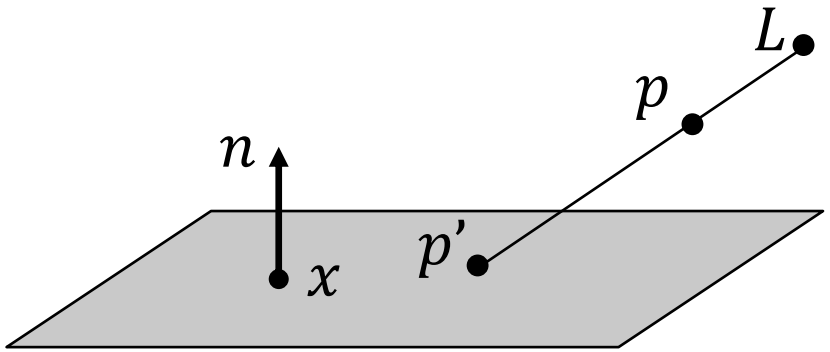
$$\begin{pmatrix} p'_x \\ p'_y \\ p'_z \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix}$$

Draw projected geometry with a darker shade of the ground plane colour

Cheap Shadow Projection

- Find p' given p, L, and plane equation
- Can we derive a homogeneous transformation matrix that projects point p onto ground plane point p' ?
  - Plane defined with normal, n, and point on plane, x





From the plane equation, let $D = -n^T x$, i.e.,

$$n^T p' + D = 0.$$

Write the parameterized ray from the light through p as

$$p' = L + t(p - L)$$

and solve for t such that p' is on the plane,

$$n^T(L + t(p - L)) + D = 0.$$

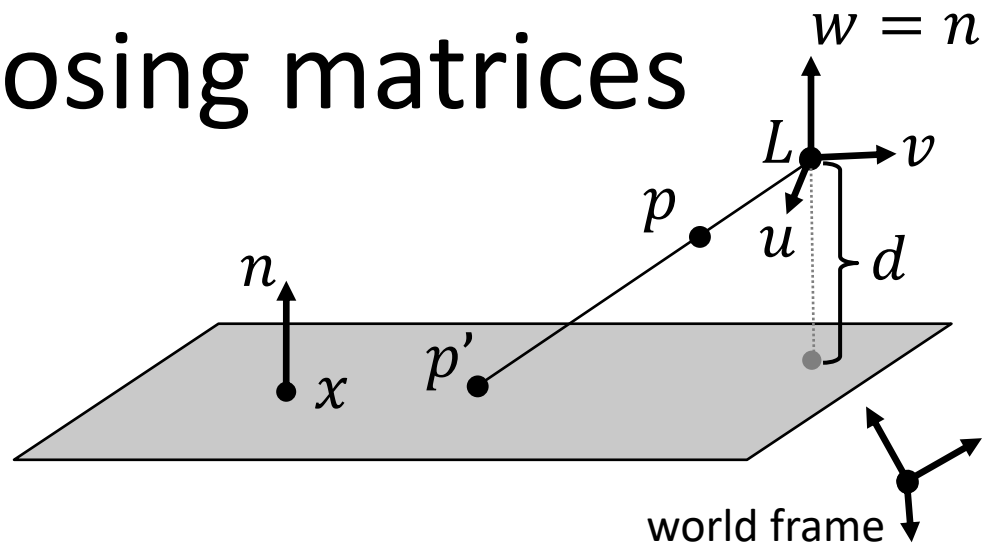
Solving for t gives $t = \frac{-(D+n^T L)}{n^T(p-L)}$. Thus,

$$p' = L + \frac{-(D + n^T L)(p - L)}{n^T(p - L)}.$$

We can tease this apart into a linear part, translation, and perspective divide…

But there is an easier way!

Build the projection by composing matrices



- Build a viewing coordinate frame at the light

$$V = \begin{pmatrix} u & v & w & L \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

  - Look in negative direction of plane normal, i.e, $w = n$
  - Construct arbitrary orthogonal u and v frame axes
- Compute distance from light to plane $d = n \cdot (L - x)$
- Recall perspective projection matrix, and the choice of a and b (previously, to preserve near and far plane)

$$P = \begin{pmatrix} d & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & a & b \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

  - What values for a and b to project points onto the plane?

- Compose transformations to project points onto the plane in world coordinates

$$p' = VPV^{-1}p$$

- Do we need to worry about left right top bottom for a frustum? Why not?
- What happens to points p that are below the plane?

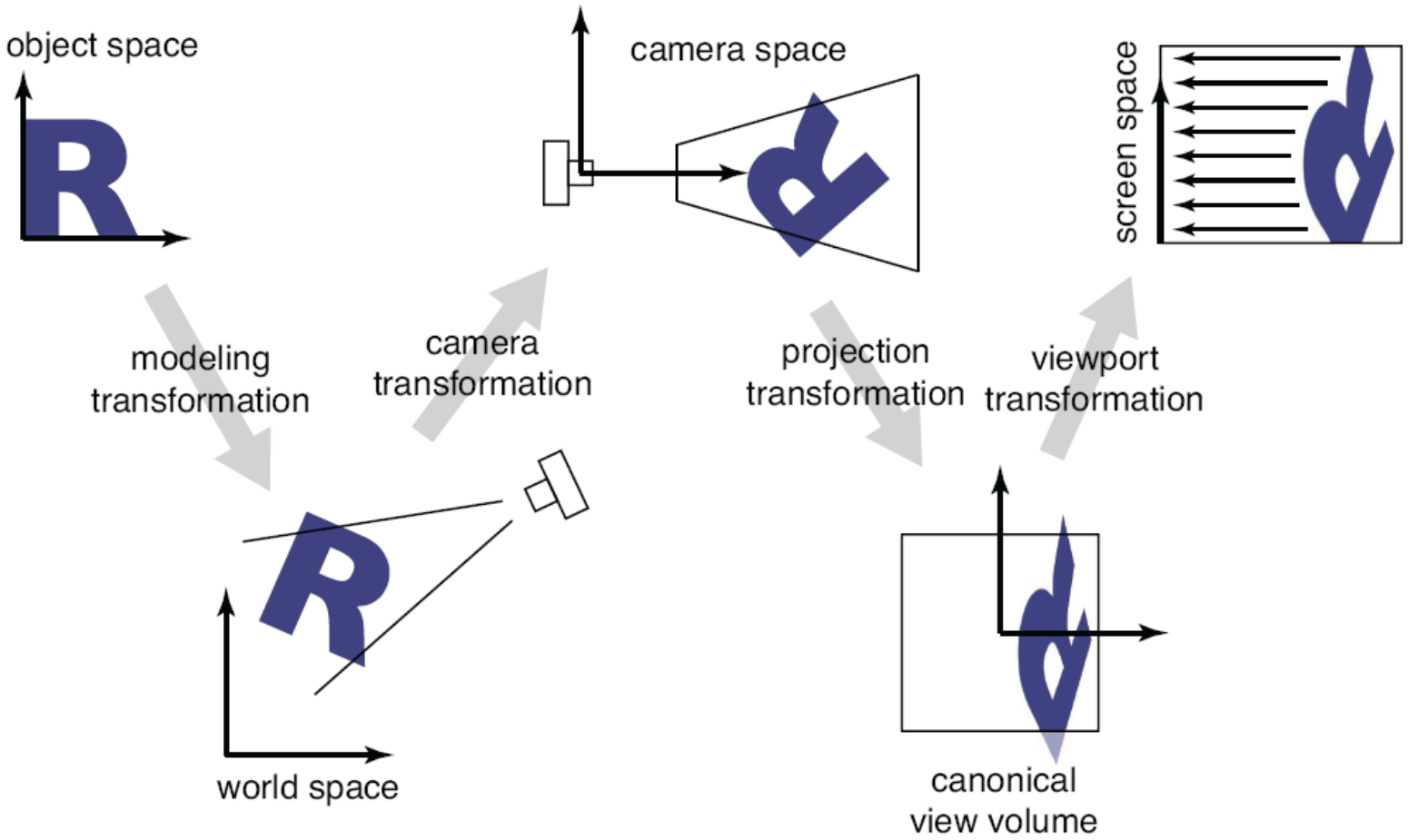
SHADOW MAPS

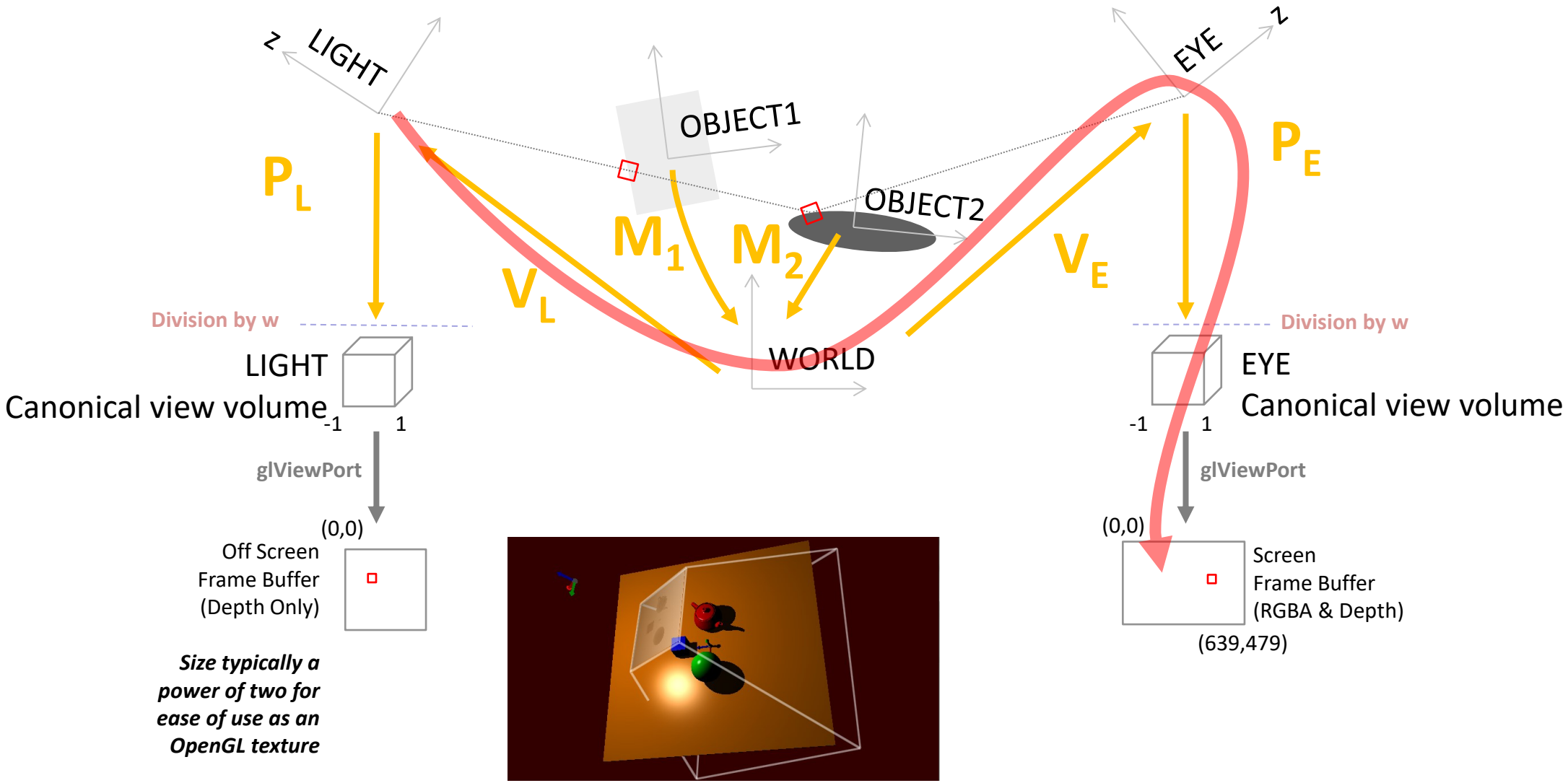
Shadows by Shadow Maps

- Surface is only illuminated if nothing blocks its view of the light
- Need to check if anything is occluding
- Object order rendering
  - ***Can draw the view from the light!***
  - Store depth information for the closest surface at each pixel (texel)
  - Compare this depth (distance) information when processing fragments (i.e., pixels) visible in the camera view

Invertible Pipeline of Transformations

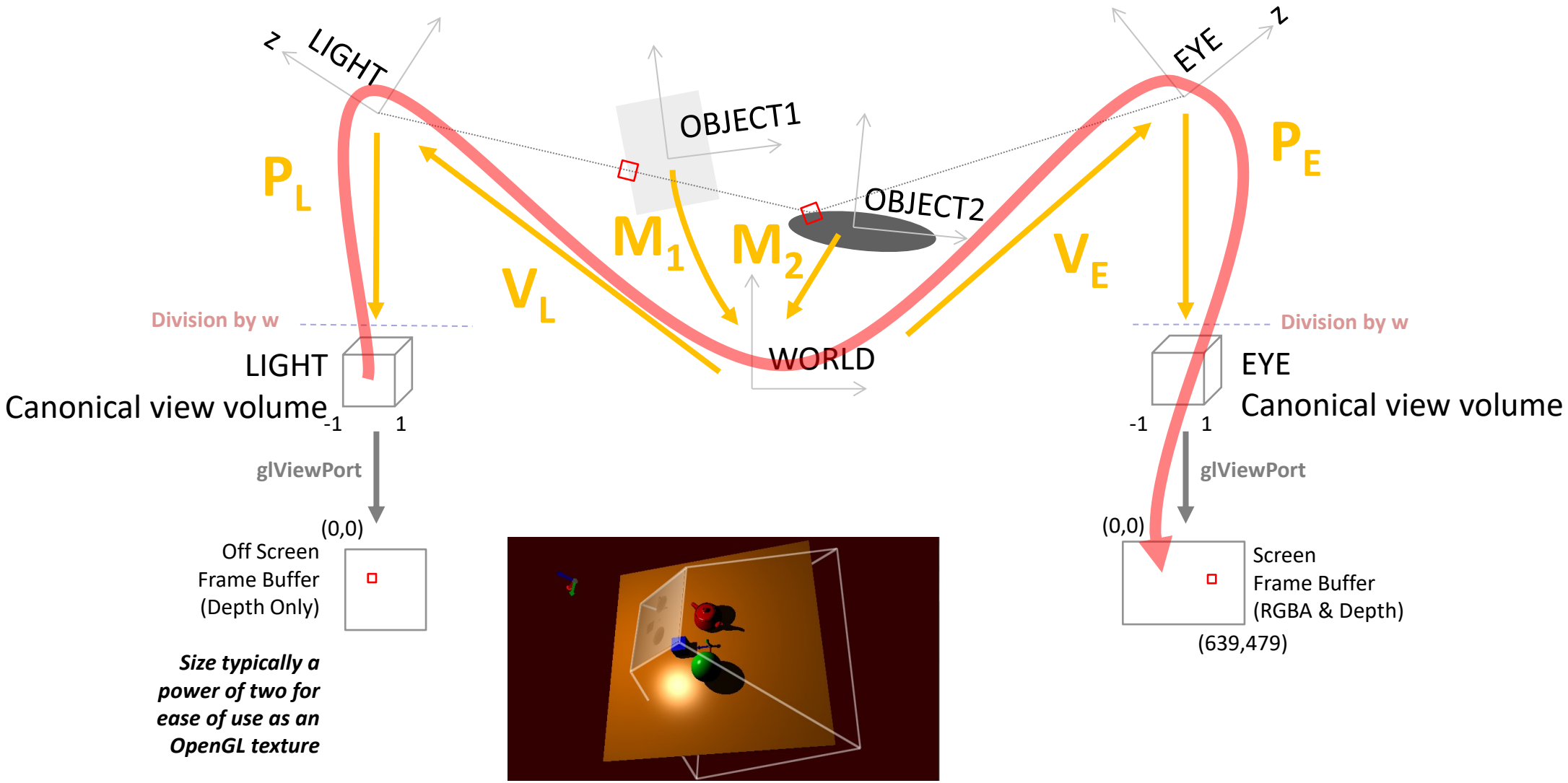
- With depth stored in screen space, recall that all transformations are invertible!





Can draw the light's camera frame

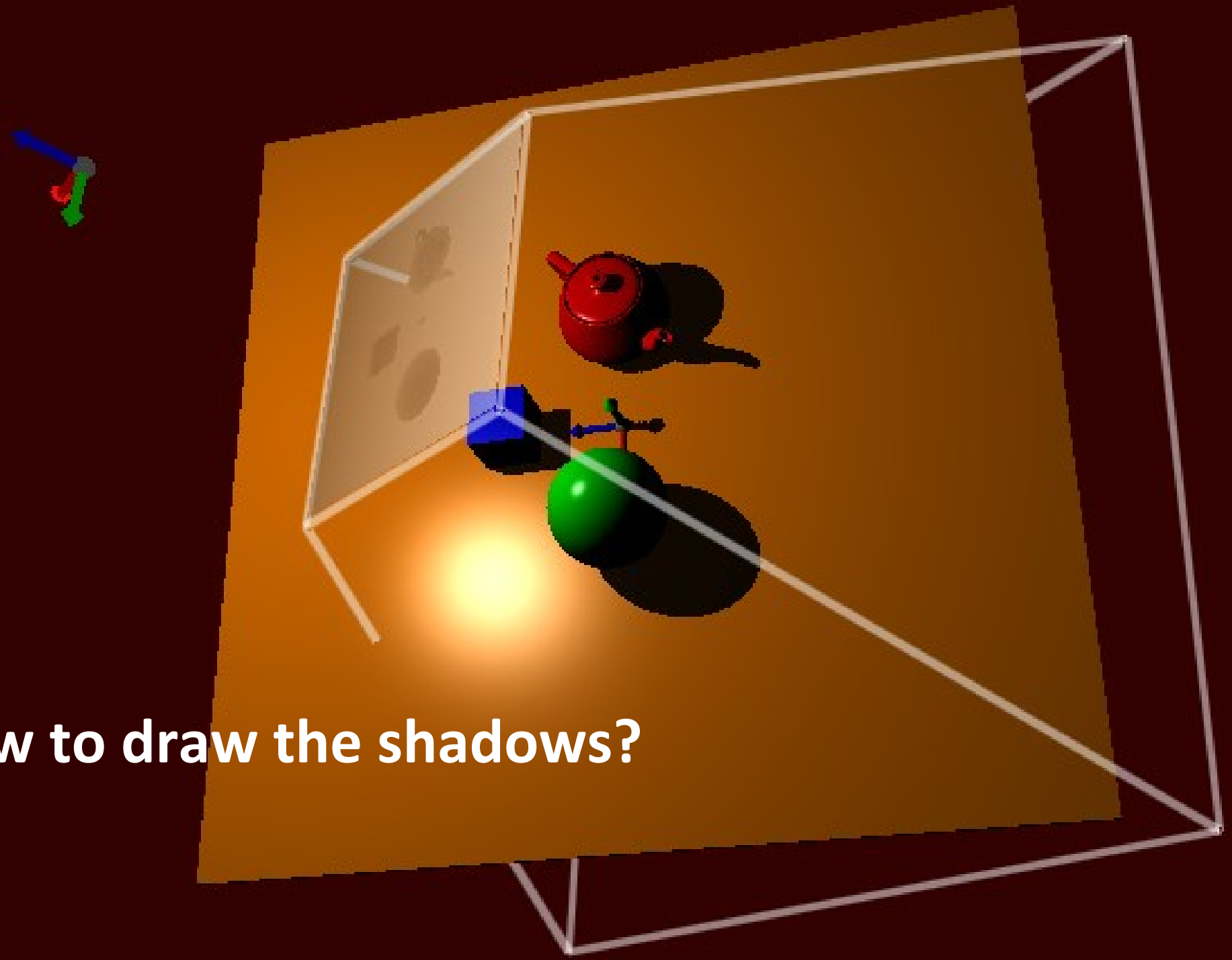
Use V_L^{-1} as the modelling transformation while drawing axis



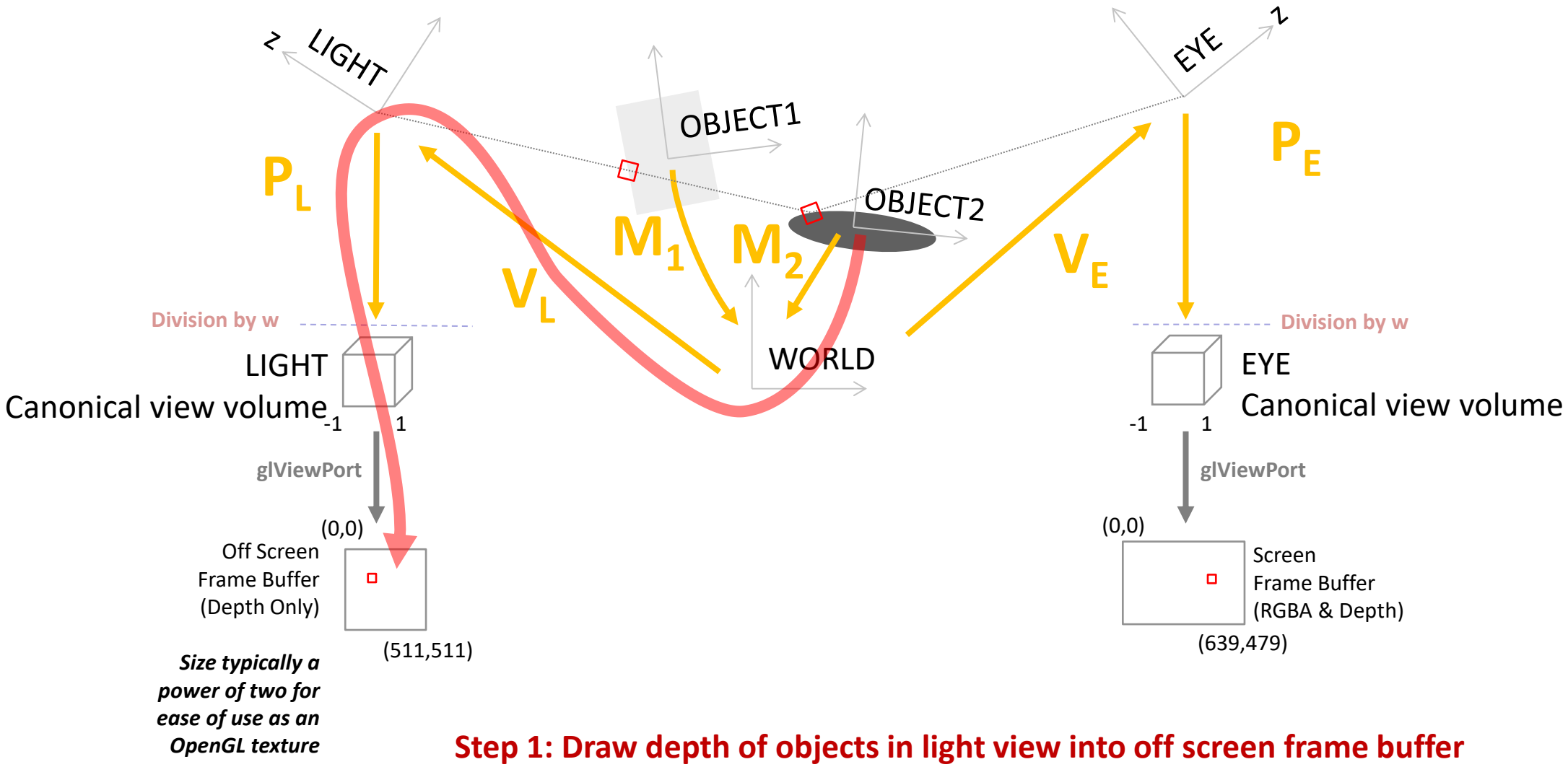
Size typically a power of two for ease of use as an OpenGL texture

Can draw the light's camera frustum

Use $V_L^{-1}P_L^{-1}$ as the modelling transformation while drawing a [-1,1] cube

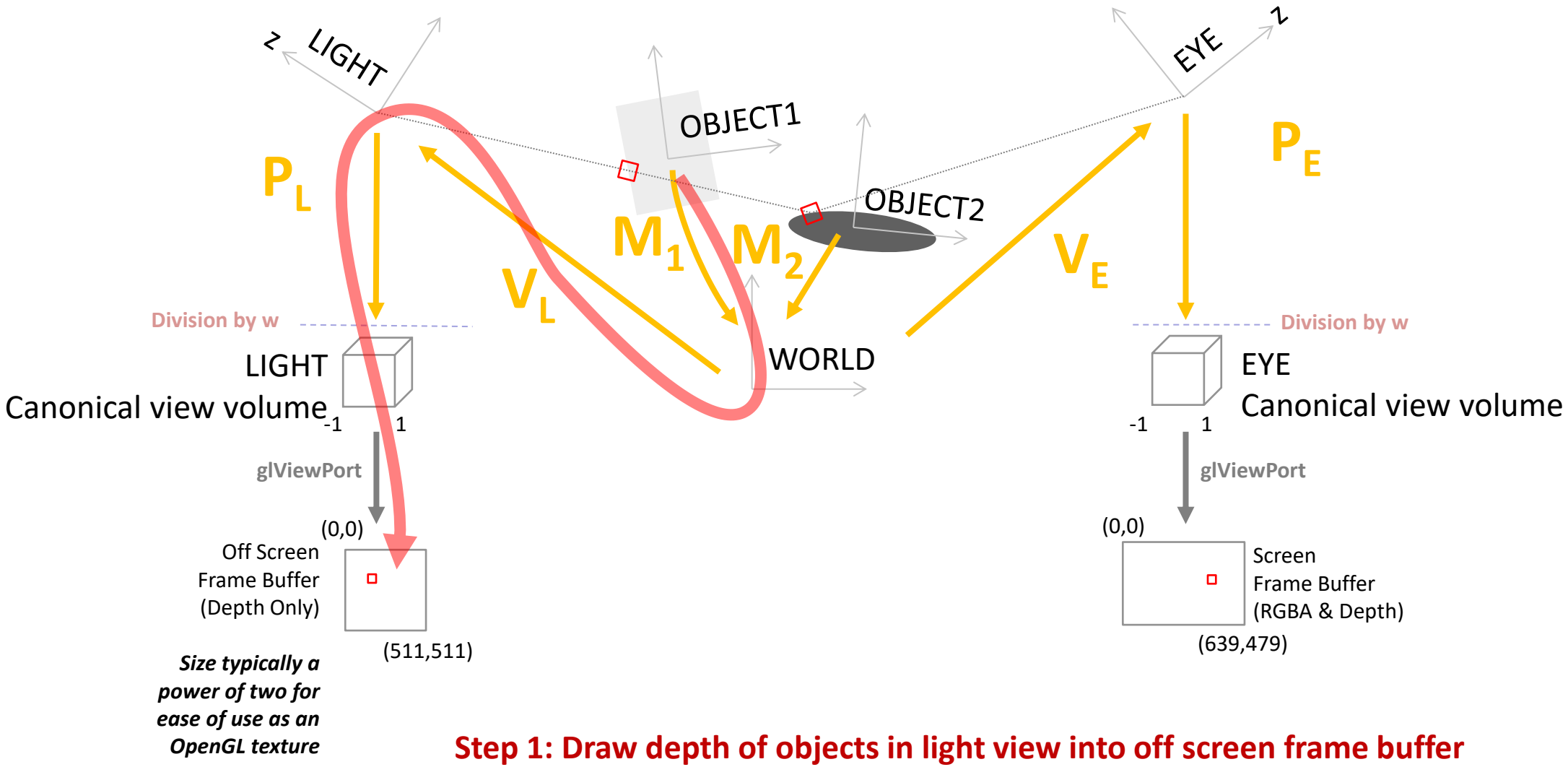


How to draw the shadows?



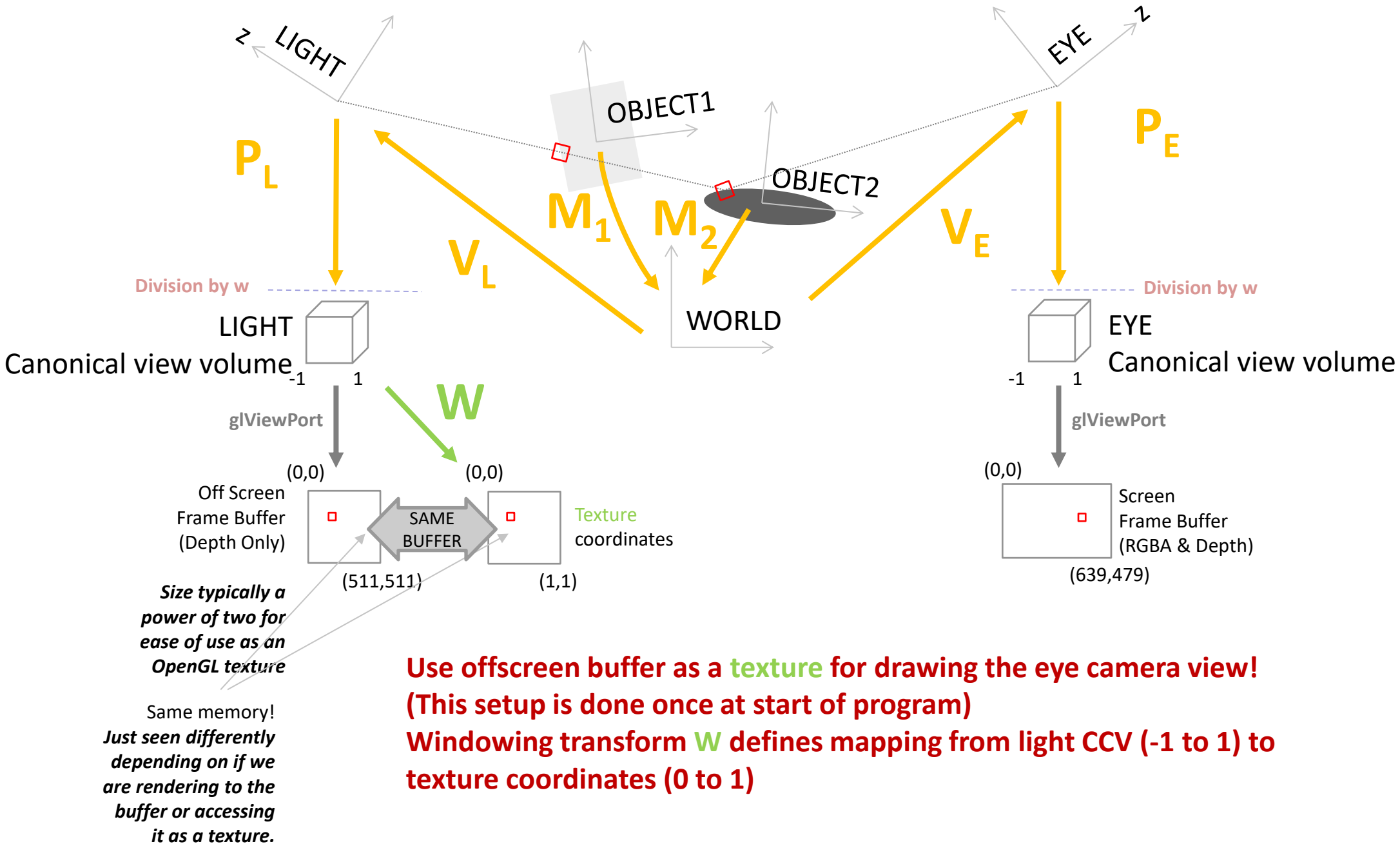
Size typically a power of two for ease of use as an OpenGL texture

Step 1: Draw depth of objects in light view into off screen frame buffer

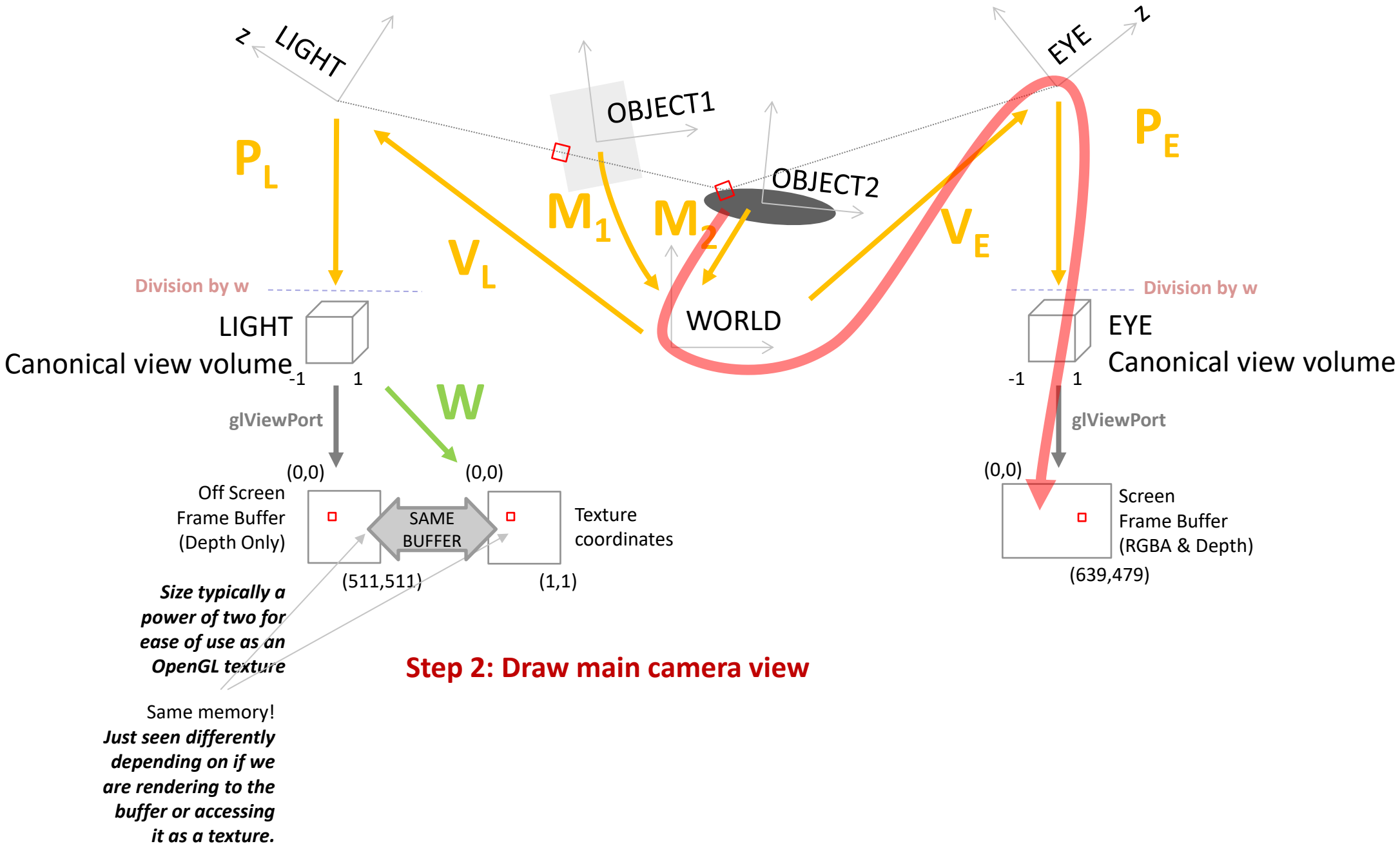


Size typically a power of two for ease of use as an OpenGL texture

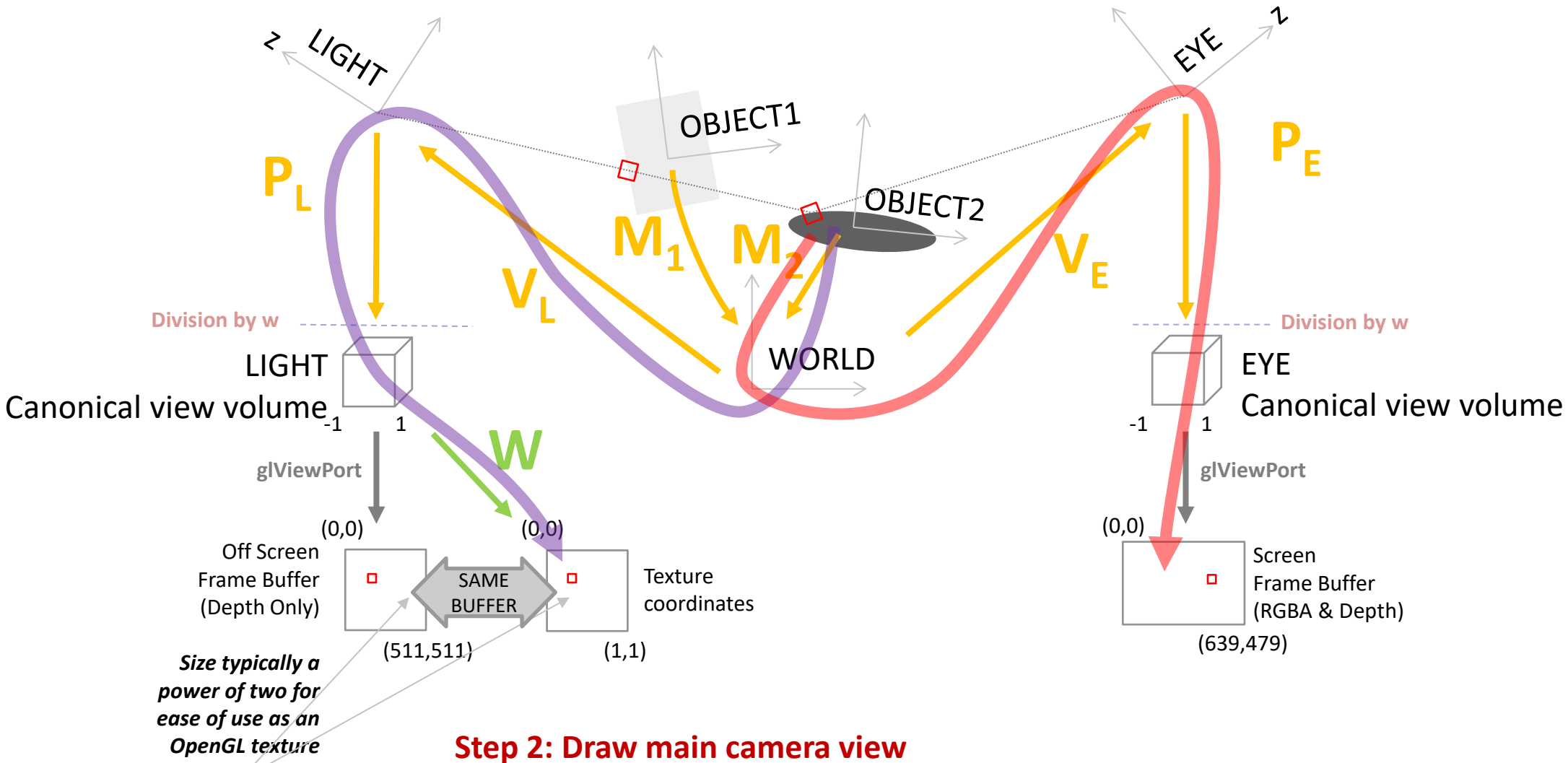
Step 1: Draw depth of objects in light view into off screen frame buffer Object 1 fragment will overwrite object 2 fragment for the example pixel, because it is closer (i.e., depth test passes in light view). Offscreen buffer will record this depth.



Use offscreen buffer as a texture for drawing the eye camera view!
(This setup is done once at start of program)
Windowing transform W defines mapping from light CCV (-1 to 1) to texture coordinates (0 to 1)



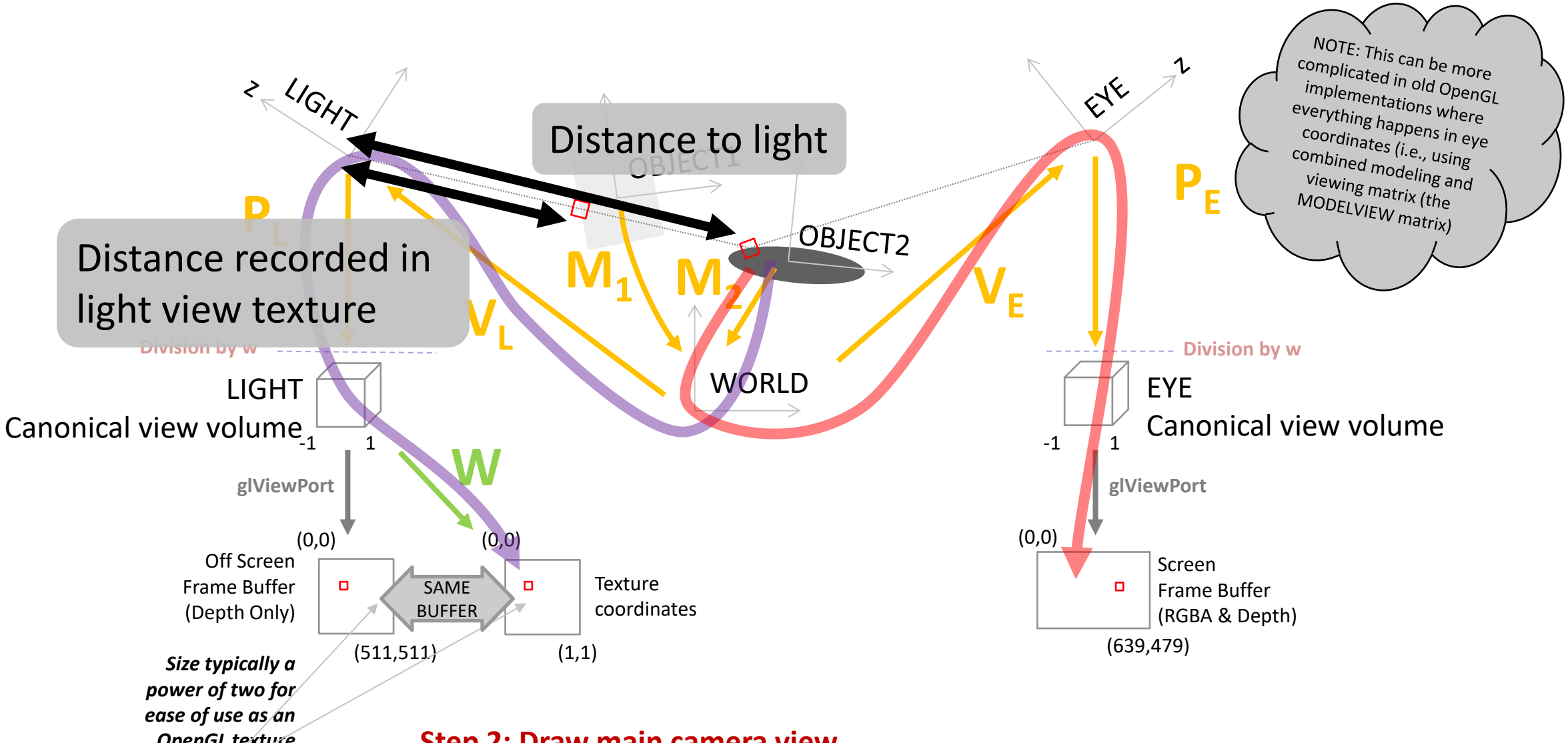
Step 2: Draw main camera view



Size typically a power of two for ease of use as an OpenGL texture

Same memory!
Just seen differently depending on if we are rendering to the buffer or accessing it as a texture.

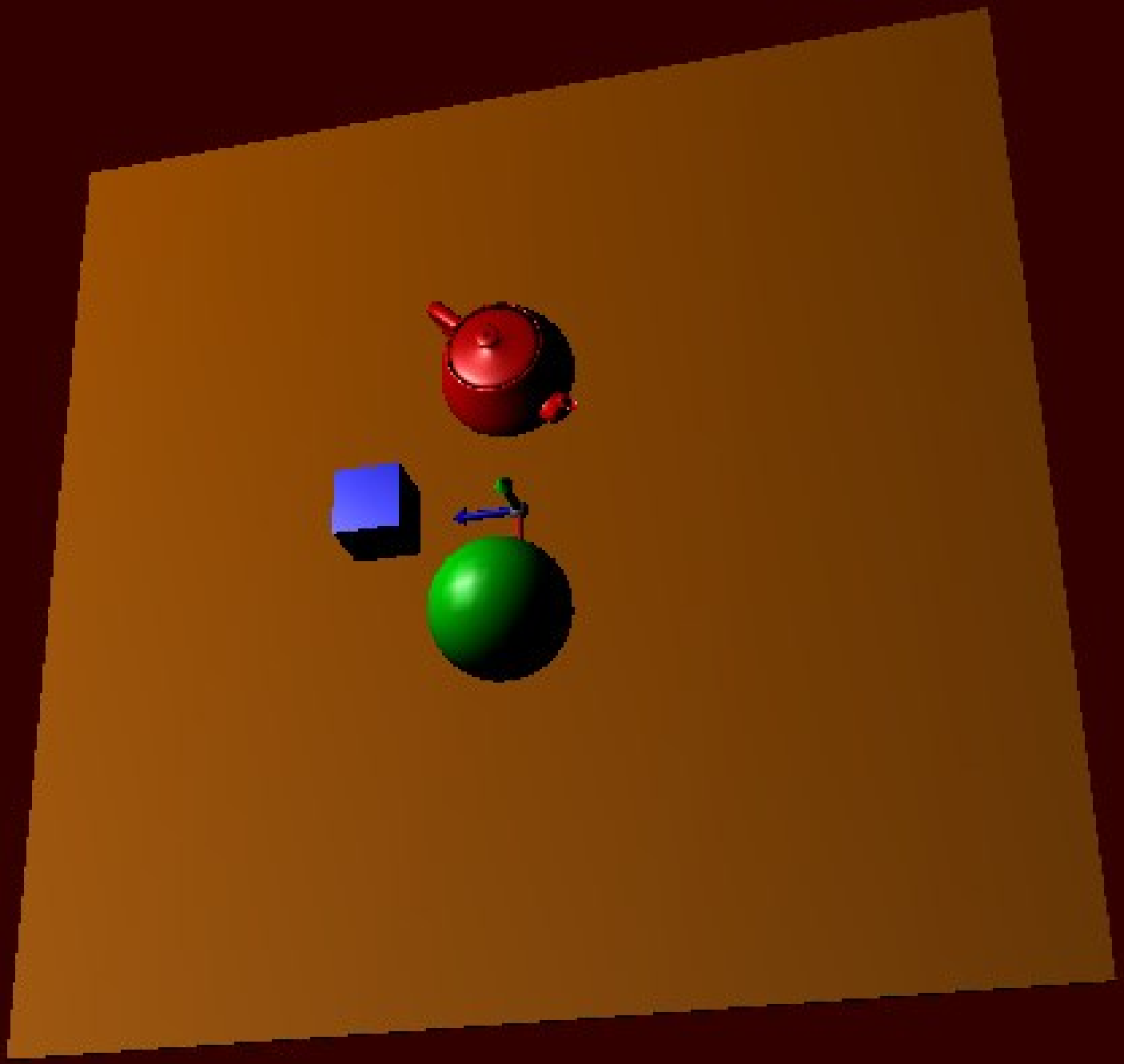
Step 2: Draw main camera view
When processing fragment of object 2, its position is also mapped to the light view depth texture. Comparing distance to light (depth) of fragment to the depth recorded in the light reveals that this fragment is in shadow!

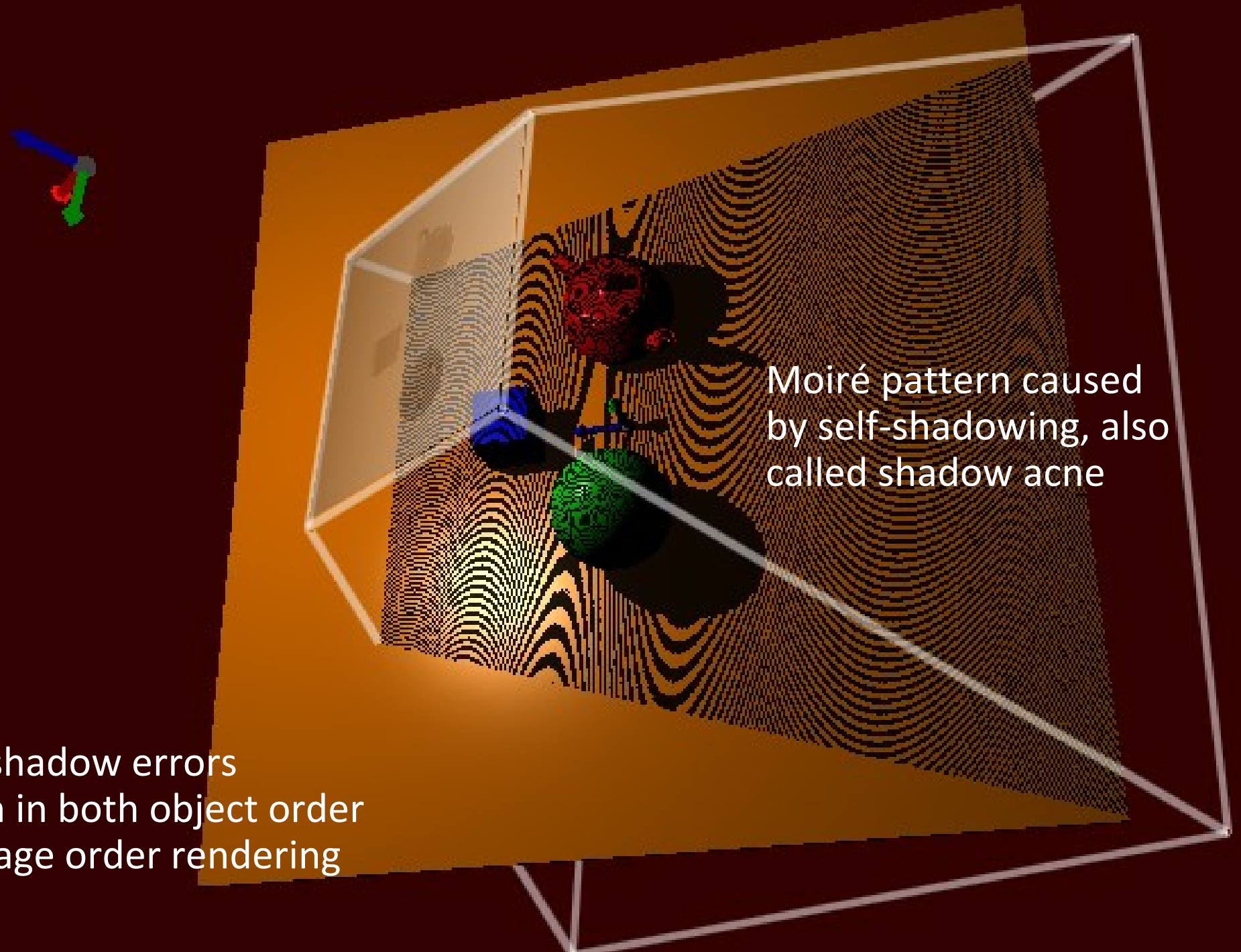


Size typically a power of two for ease of use as an OpenGL texture

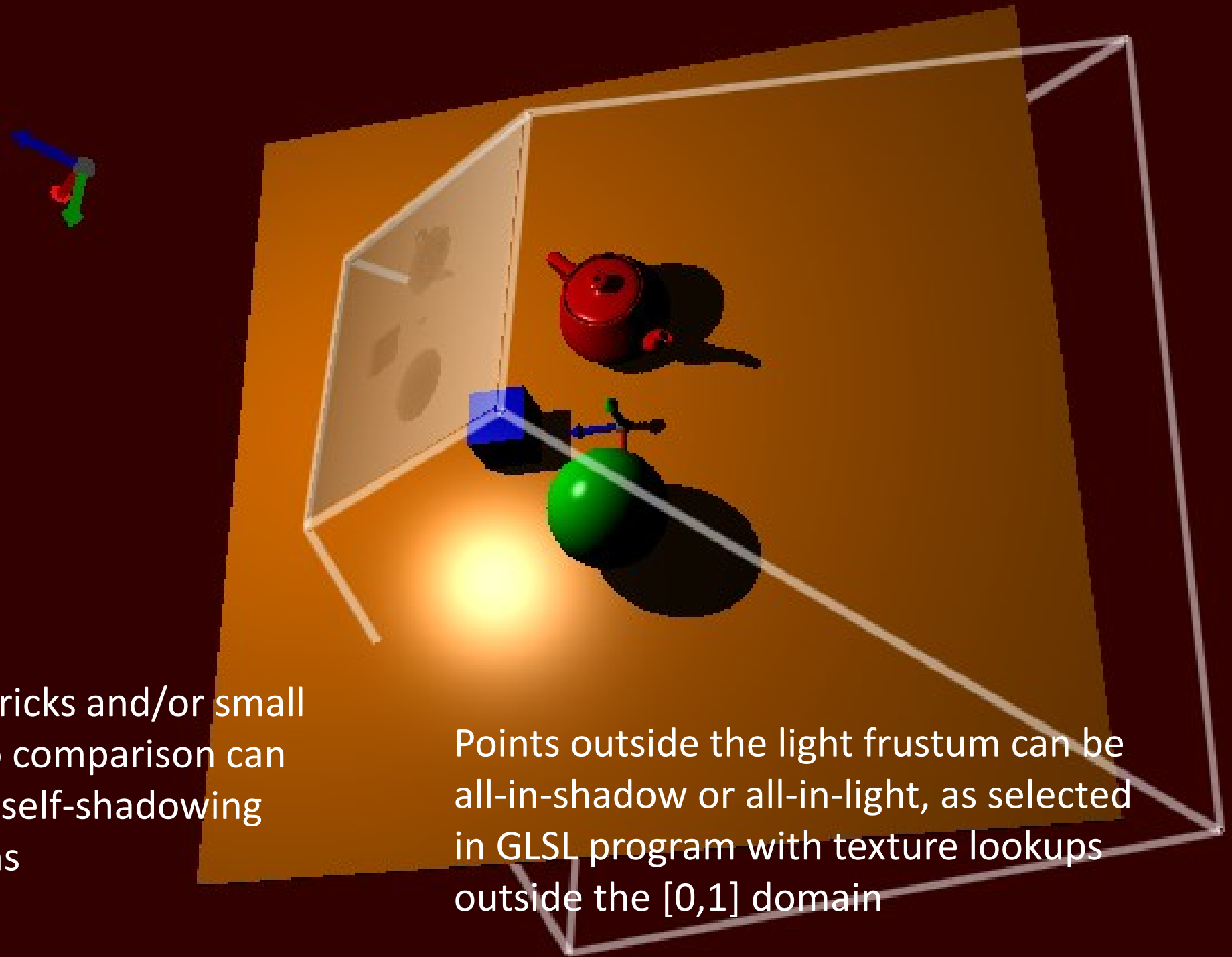
Same memory!
Just seen differently depending on if we are rendering to the buffer or accessing it as a texture.

Step 2: Draw main camera view
When processing fragment of object 2, its position is also mapped to the light view depth texture. Comparing distance to light (depth) of fragment to the depth recorded in the light reveals that this fragment is in shadow!





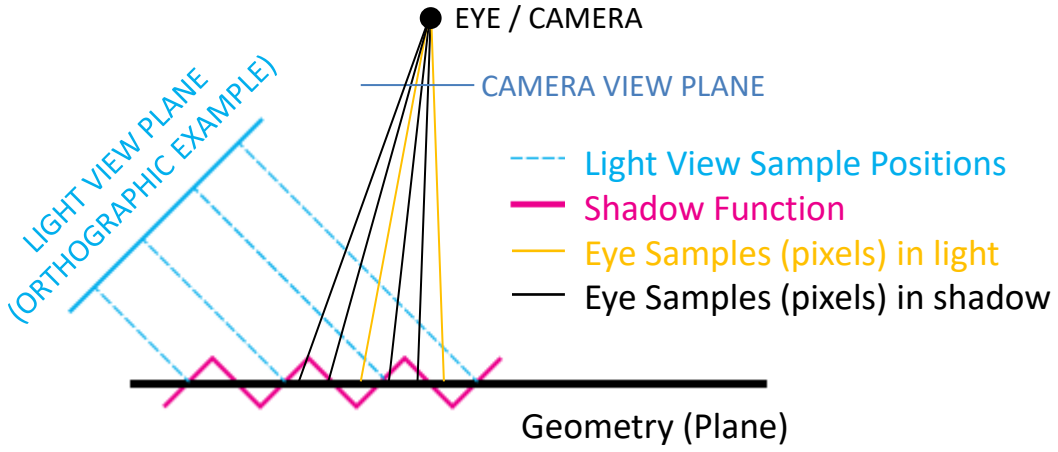
These shadow errors happen in both object order and image order rendering

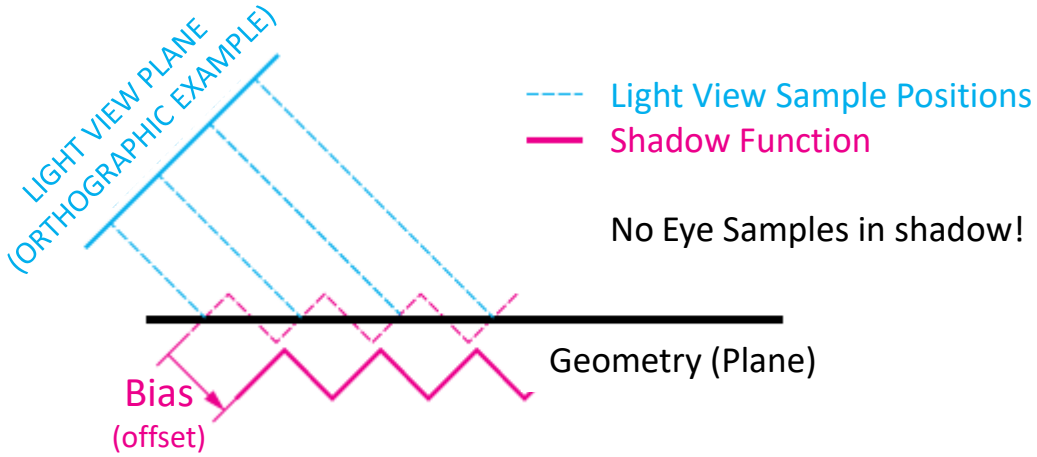


Culling tricks and/or small offset to comparison can address self-shadowing problems

Points outside the light frustum can be all-in-shadow or all-in-light, as selected in GLSL program with texture lookups outside the [0,1] domain

Offset (bias) to fix shadow acne



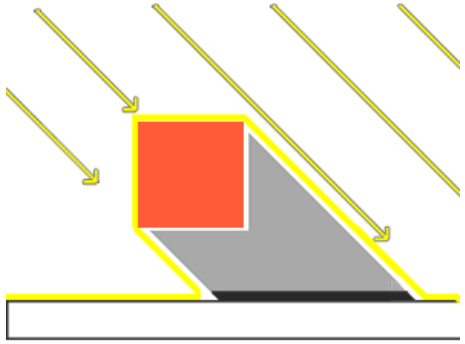


- Light view samples (texels) have a finite size
  - Distance-to-light shadow function is a piecewise constant “staircase”
  - Some camera view samples (pixels) will be farther from light, some closer
- Arbitrary small offset can do the job in many cases
- Can vary the bias based on the slope of the surface too!

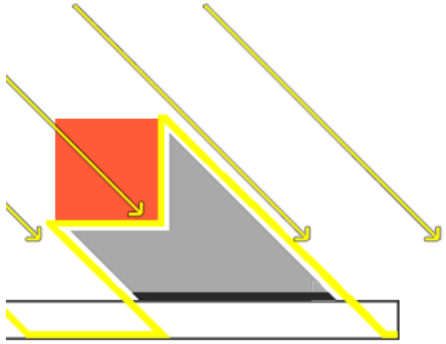
Culling to fix shadow acne

Recall front face typically defined by screen space ***counterclockwise*** order of triangle vertices. This was mentioned briefly in pipeline slides and will be discussed again in material on meshes.

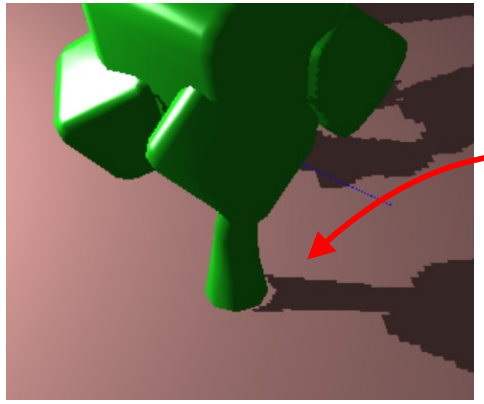
- With solid objects, ***back face culling*** can reduce rasterization
  - Can't use this trick for thin objects, i.e., want to see both sides
- In shadow map view, can use ***front face culling*** to reduce self-shadowing with solid geometry
  - Back faces still cast shadows! But watch out for light leaks!



no culling



front-face culling



GLSL IMPLEMENTATION

GLSL Lighting Computations

- Vertex program ***typically*** computes vertex in eye coords as needed for lighting
  - Point-based lights ***historically*** stored in eye coords, to compute the light direction
  - This is also needed for Phong and Blinn-Phong lighting to compute the viewing direction
  - ***Can also choose to use world coordinates*** if we know view and light position in world coordinates
- Here, an example ***vertex program***, that computes position in camera coordinates and passes ***out*** vec3 to be interpolated (i.e., across a triangle) and used to compute output for each pixel in the ***fragment program***

```
in  vec4 in_position;      // vertex position specified in buffer
in  vec4 in_normal;        // normal at vertex
out vec3 v_vert, v_normal; // surface fragment location and normal in camera frame
uniform mat4 P, V, M;      // Projection, view transformation, modelling transformation
uniform mat4 MinvT;        // Inverse transpose of modeling transformation for normals
…
v_vert = vec3( V * M * in_position );     // Transform vertex to camera frame
v_normal = vec3( V * MinvT * in_normal ); // Transform normal to camera frame
gl_Position = P * V * M * in_position;    // Transform vertex to CCV
```

GLSL Lighting Computations in Eye Camera Coordinates

```
in vec3 v_vert;   // surface fragment location in camera coordinates
in vec3 v_normal; // normal of fragment in camera coordinates
```

Given the fragment location in the camera frame, diffuse lighting can be computed with the following lines in a ***fragment program***

```
vec3 L = normalize( lightPosition.xyz – v_vert );
vec4 Ld = material_kd * lightIntensity * max(dot(v_normal,L), 0.0);
Ld = clamp(Ld, 0.0, 1.0);
```

For specular component, note that the view direction is easy!

```
vec3 view_direction = normalize ( -v_vert );
vec3 h = normalize( view_direction + L )
vec4 Ls = material_ks * lightIntensity * pow(max(dot(h, v_normal),0),p)
```

Shadow Map lookup

```
uniform sampler2D u_shadowMapValues;
uniform sampler2Dshadow u_shadowMapTest;
```

- **u_shadowMapValues** provides depth texture rendered from the light view
- **`u_shadowMapTest`** provides value in [0,1] based on depth test (see next slide)
- These are uniforms! To what should they be set?
  - Can have an effectively unlimited number of **textures** in OpenGL, each has an ID
    - Number of textures you have is only effectively limited by memory
  - There are many **texture units** in OpenGL, each has a number
    - A limited resource, minimum 16 in OpenGL 3.0, 80 in OpenGL 4.0, perhaps hundreds on a modern GPU
  - The texture unit does the work of sampling the texture (e.g., looking up and mixing values)
  - The **sampler uniform** must be told **which texture unit** to use
  - The **texture unit** must likewise be told which **texture ID** to use for sampling

Shadow Map lookup

- Need to compute lookup **pos** in texture for the fragment position
  - This comes from input positions, mapped to light's canonical view volume followed by a windowing transformation, **pos = W * PL * VL * M * in_position** (but do this efficiently with precomputed matrix and with transformation in the vertex program)
  - For correct texture coordinates, must **divide through by w**, i.e., **pos = pos.xy / pos.w**

```
float distanceFromLight = texture( u_shadowMapValues, pos.xy ).r
float shadow_factor = texture( u_shadowMapTest, pos.xyz )
```

- Option 1: Compare `distanceFromLight` of what the light saw to the distance to light from the fragment currently being shading (which is at distance `pos.z`).
  - Watch out for self shadowing (i.e., may want to include an offset)!
- Option 2: Better to use `shadow_factor` and let the sampler figure out the shadow with "percentage closer filtering" (seem more on this later)
  - Note you still need to include some offset in pos.z to avoid self shadowing!

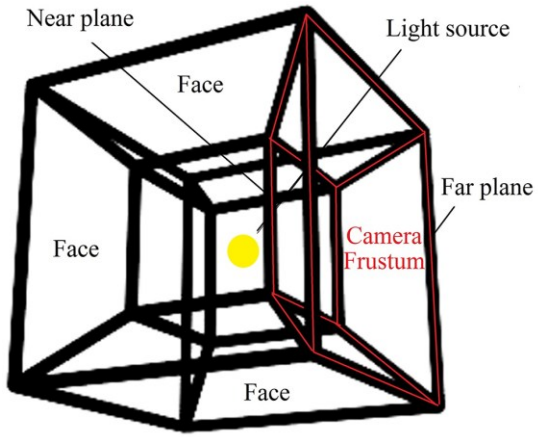
IMPROVING SHADOW MAPPED SHADOWS

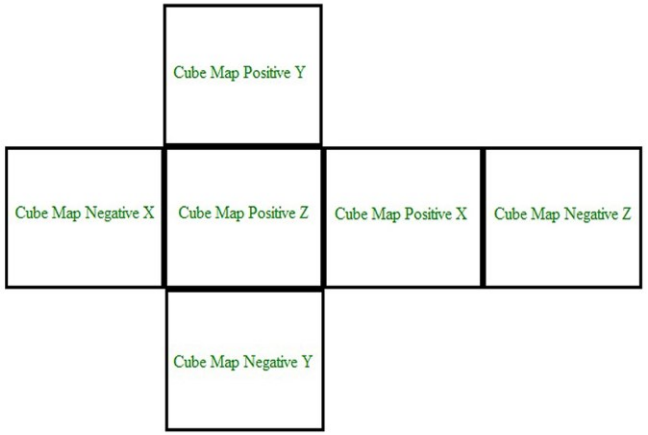
Choosing the light view frustum

- FOV of light view greatly affects appearance of shadows
  - Increasing FOV spreads shadow map resolution over larger area
- Can choose appropriate near / far / field of view for light
  - nf: identify min and max z of geometry in light coordinate frame
  - $lrtb$: identify min and max of x and y of geometry after projection matrix, but before windowing transform to CCV
  - Can use convex hull or bounds of geometry, or other approximations
- Only need to include shadow casting objects (if known)

Shadow Cube-Map for Point lights

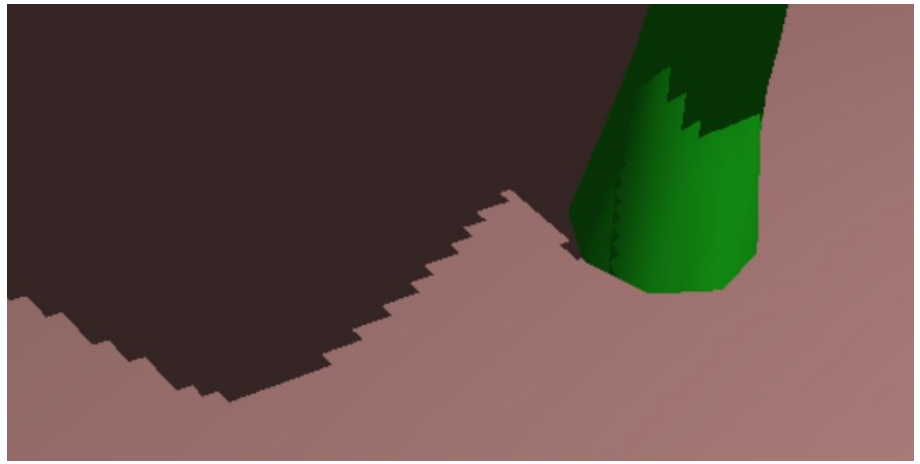
- A point lights casts shadows in all directions
  - Render 6 light-view depth maps
  - One for each cube face
  - Setup cube map texture
  - Use `samplercubeshadow` for the cube map lookup when checking if fragment from camera view (i.e., pixel) is illuminated

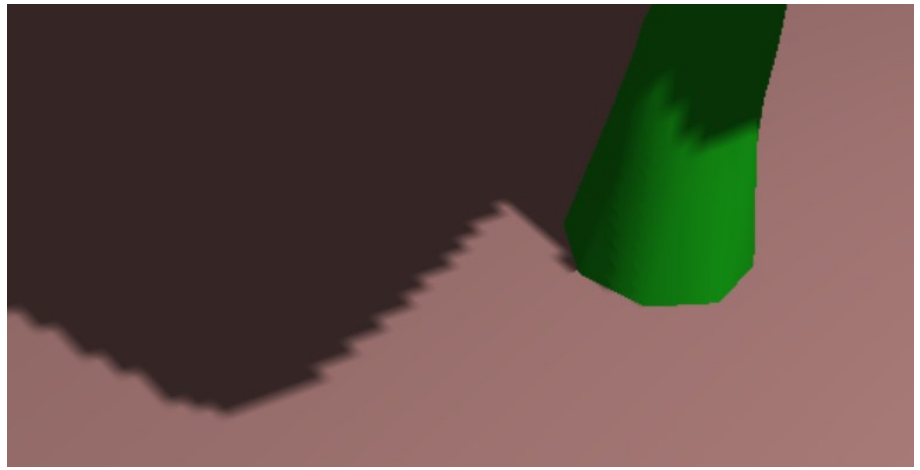




Softer Shadows with Percentage Closer Filtering

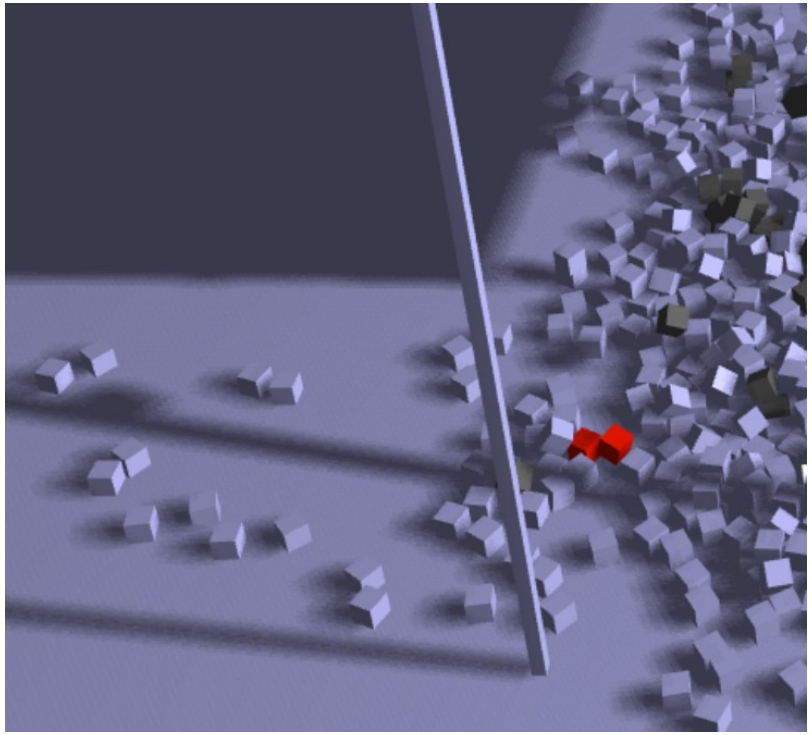
- A sampler can use different filtering
  - NEAREST takes the closest sample, while LINEAR interpolates 4 closest
- For sampler2Dshadow, LINEAR filtering does **percentage closer filtering**
  - Hardware does a compare for each sample, and interpolates the result





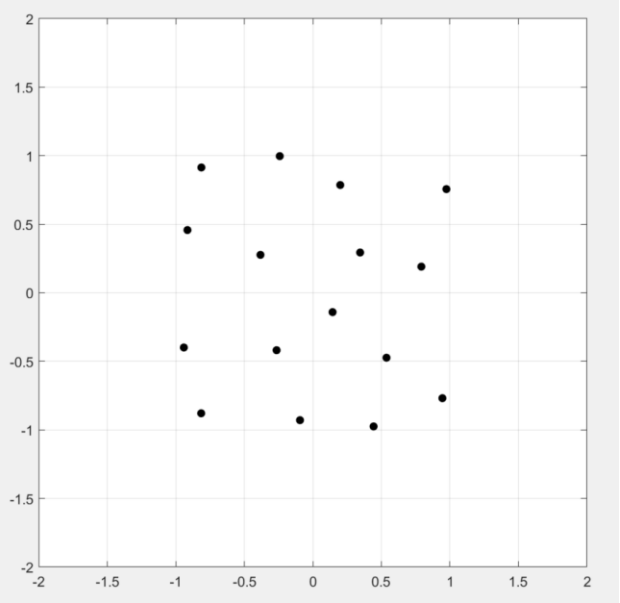
Softer Shadows with Extra Samples

- Percentage closer filtering with additional shadow map lookups provides additional softening



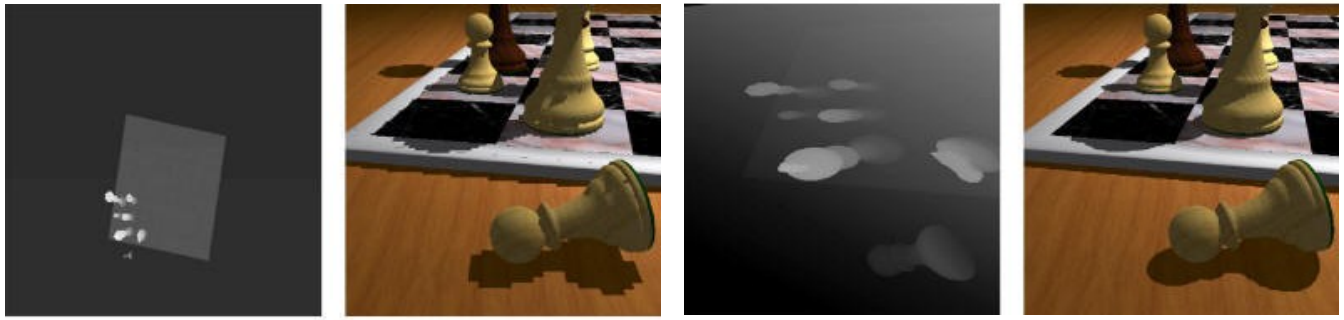
```
 vec2 poissonDisk[16] = vec2[](
   vec2( -0.94201624, -0.39906216 ),
   vec2( 0.94558609, -0.76890725 ),
   vec2( -0.094184101, -0.92938870 ),
   vec2( 0.34495938, 0.29387760 ),
   vec2( -0.91588581, 0.45771432 ),
   vec2( -0.81544232, -0.87912464 ),
   vec2( -0.38277543, 0.27676845 ),
   vec2( 0.97484398, 0.75648379 ),
   vec2( 0.44323325, -0.97511554 ),
   vec2( 0.53742981, -0.47373420 ),
   vec2( -0.26496911, -0.41893023 ),
   vec2( 0.79197514, 0.19090188 ),
   vec2( -0.24188840, 0.99706507 ),
   vec2( -0.81409955, 0.91437590 ),
   vec2( 0.19984126, 0.78641367 ),
   vec2( 0.14383161, -0.14100790 )
);

float shadow = 1.0;
float scale = 700; // resolution dependent
for ( int i=0; i<16; i++ ) {
    float d = texture( shadowMap, pos.xy + poissonDisk[i]/scale ).r + bias;
    shadow -= d < pos.z ? (1.0/16.0) : 0 ;
}
```

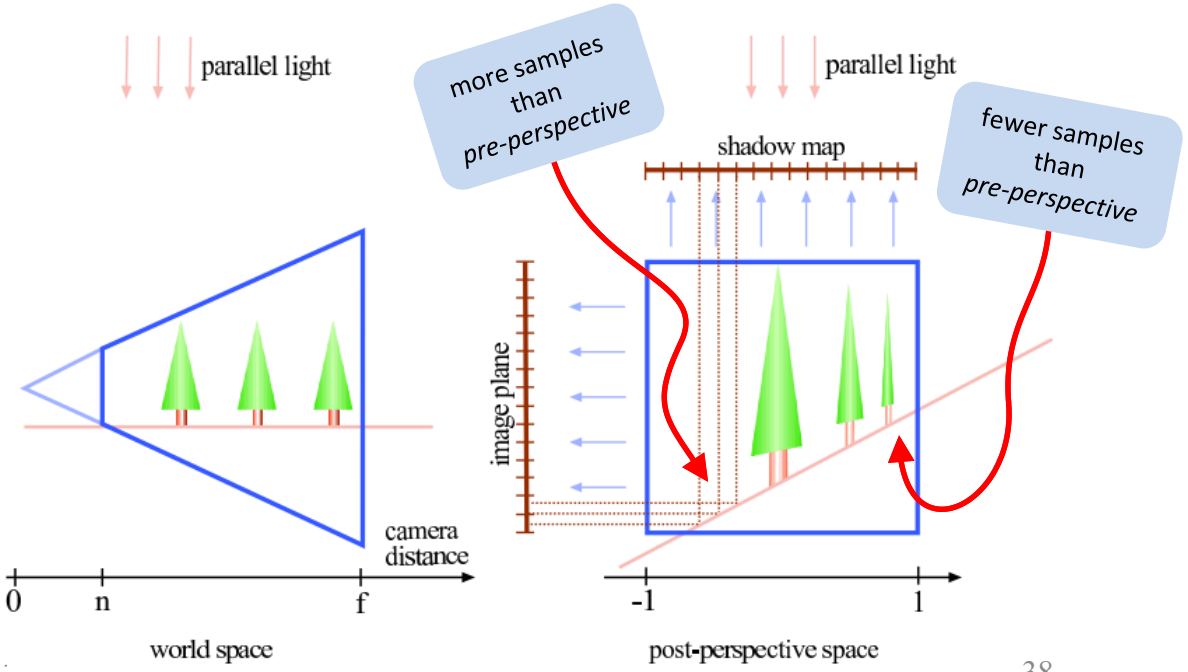


random displacements

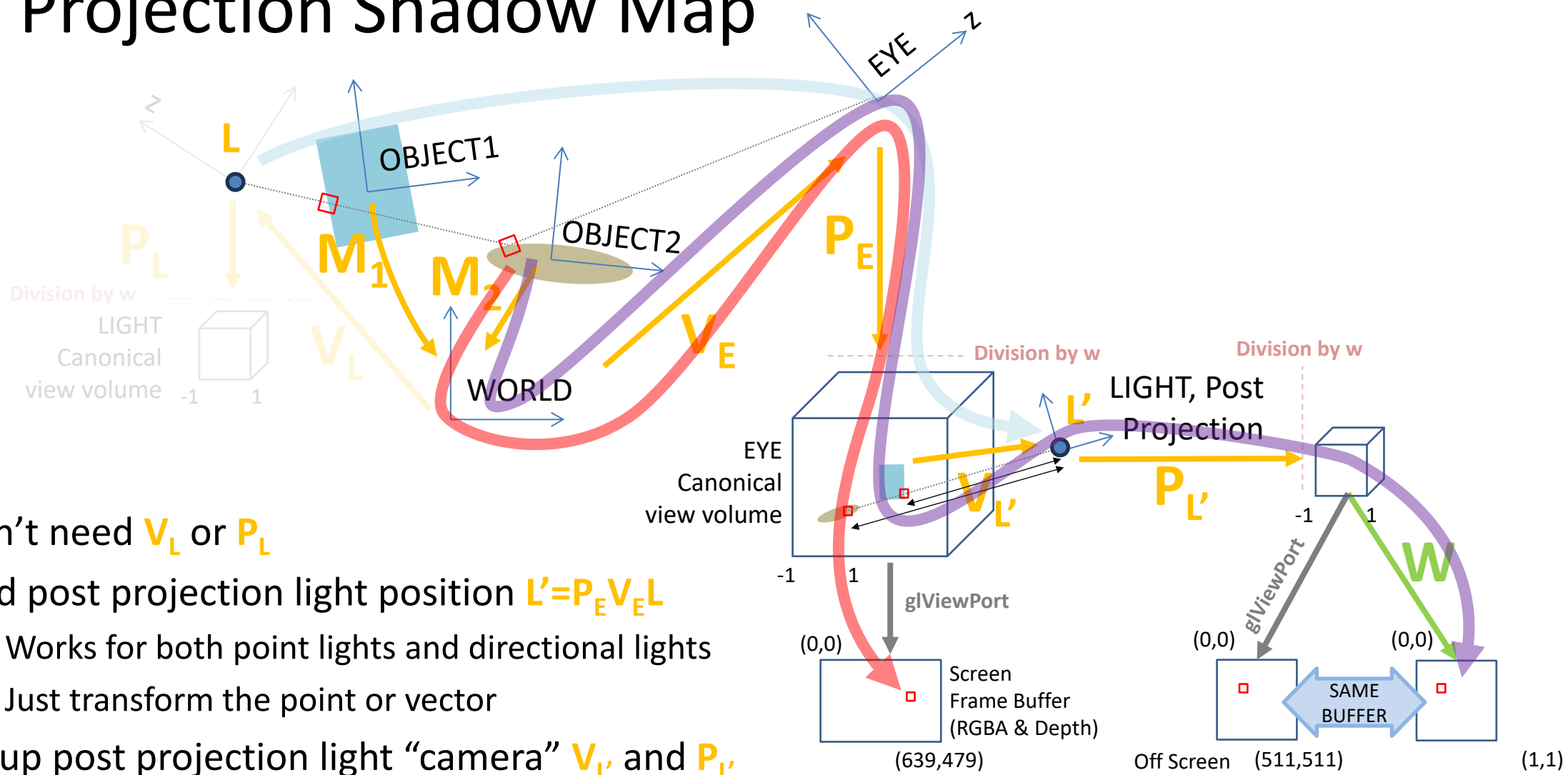
Post Perspective Shadow Maps



- Light view samples have varying size in the camera view
  - May see texels of the shadow map
- Can compute the shadow map post perspective projection
  - Set up a light camera to view geometry in CCV $[-1,1]^3$ coords
  - Lines are preserved by projection
    - Thus, this technique simply redistributes light direction sample density



Post Projection Shadow Map

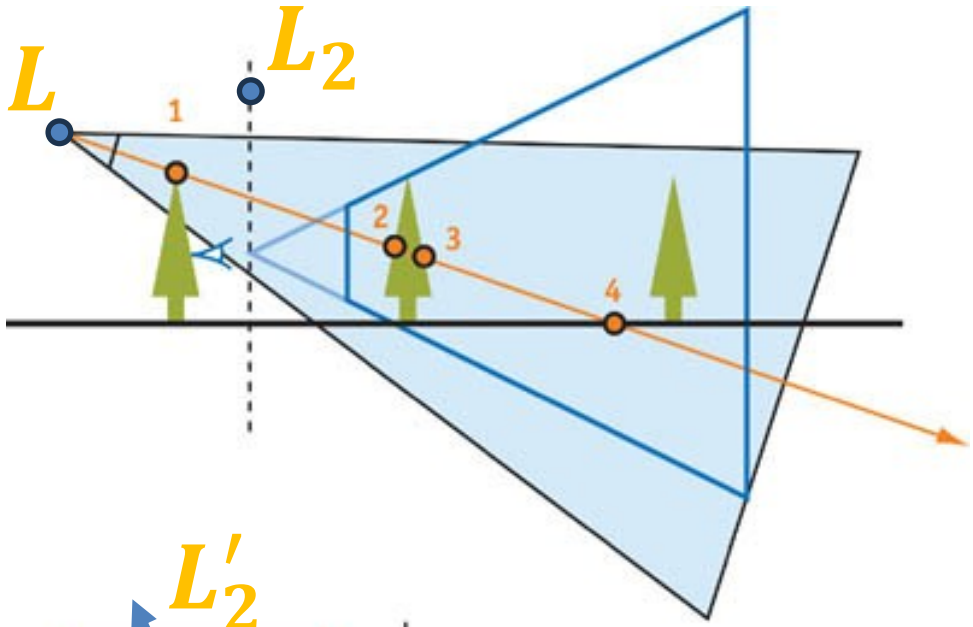


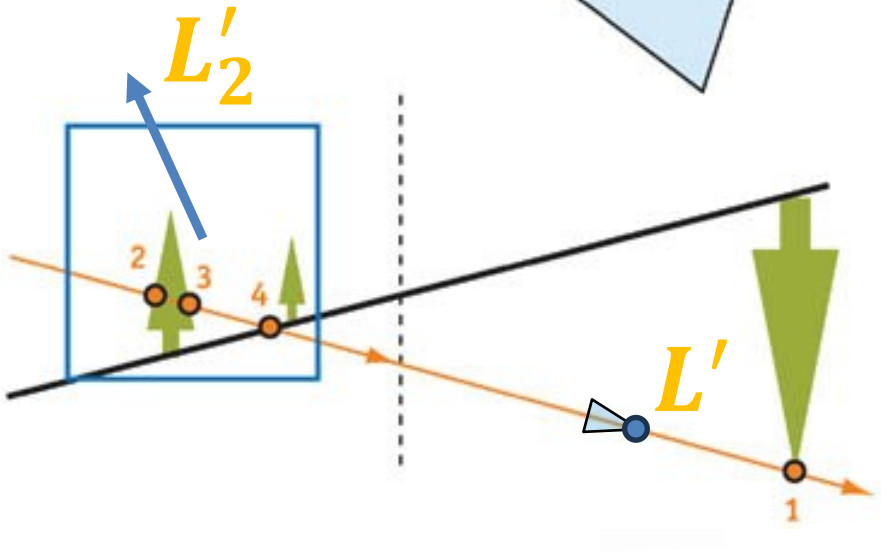
- Don't need V_L or P_L
- Find post projection light position $L'=P_E V_E L$
  - Works for both point lights and directional lights
  - Just transform the point or vector
- Setup post projection light "camera" $V_{L'}$ and $P_{L'}$
  - Render depth from light, post projection
  - Compare depths post projection

In this scenario, what colour would we see in the small red box in the screen buffer above?

Post Perspective Shadow Map Challenges

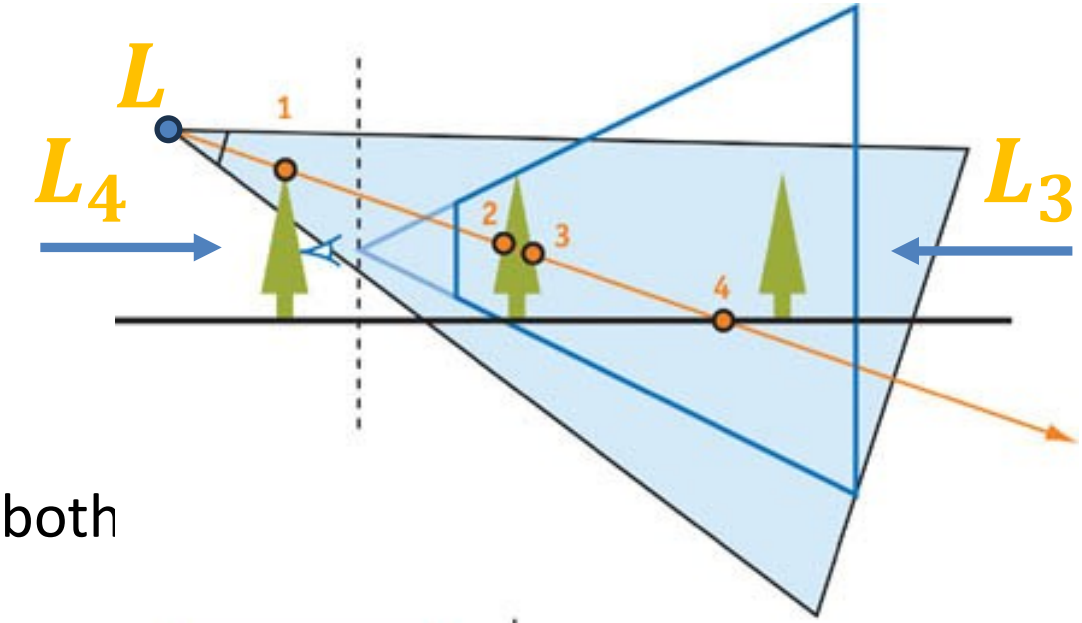
- Point light behind viewer become point lights beyond the $n + f$ plane
  - They require extra attention!
  - Light view post-perspective from behind the $n + f$ plane must inverse depth test to keep farthest fragments!
  - Objects behind camera in front of light behind camera becomes tricky
- Point light in the xy plane going through the origin becomes a directional light
  - Careful with direction of light vector (typically specify directional lights as vector from source to scene)

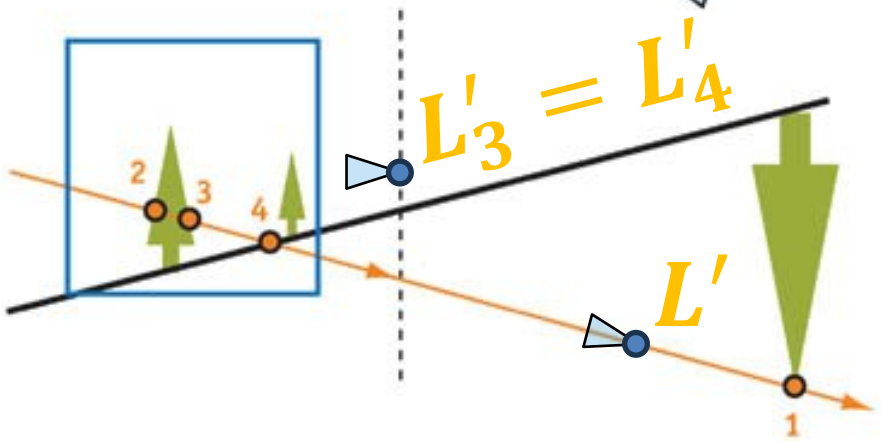




Post Perspective Shadow Map Challenges

- Directional lights become a point lights on the $n + f$ plane
  - Consider L_3 and L_4
  - Both have same post-perspective position, different homogenous representations
  - Same light projection post-perspective for both
- Light L'_3 keeps closest fragments
- Light L'_4 inverses depth test and must keep farthest fragments
  - Objects behind camera that block the directional light become tricky





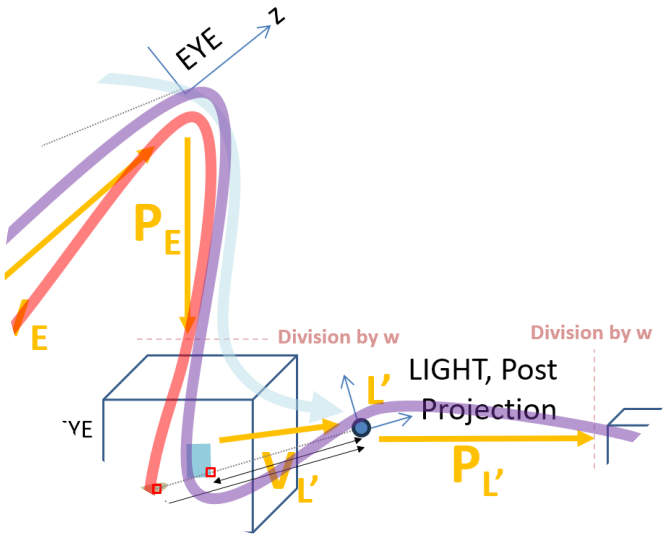
Two Projections?

- What happens when you combine two projections?
  - Consider the projection and windowing transformation P_1 given by `glm.frustum(-1,1,-1,1,1,3)` taking parameters $l\ r\ b\ t\ n\ f$

$$P_1 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -2 & -3 \\ 0 & 0 & -1 & 0 \end{pmatrix} \quad P_2 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -2 & 0 & -3 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 \end{pmatrix}$$

  - What can you say about P_2?
  - What can you say about the product

$$P_1 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -2 & 0 & -3 \\ 0 & 3 & -2 & 0 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

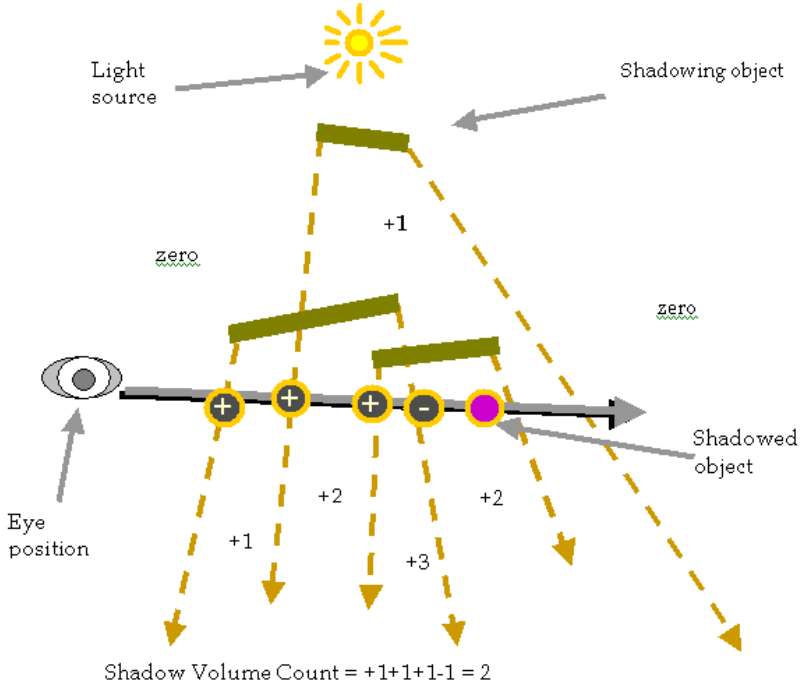


ALTERNATIVES / MORE INFORMATION

Shadow Volumes

- Pixel-perfect shadows in screen space
- Use a special buffer, the ***stencil buffer***, and draw polygons associated with all silhouette edges
- Count number of times you step into or out of shadow

[Kuttuva]
[Everitt, Kilgard]



References and More Information

- FCG 11.7 (VERY brief mention of shadow maps)
- Efficient Real Time Shadows Book, Eismann, 2.1, 3, 4.1
  https://graphics.tudelft.nl/Publications-new/2013/EASVW13/a18-eisemann.pdf
- Shadow mapping tutorial (old but still relevant)
  http://www.opengl-tutorial.org/intermediate-tutorials/tutorial-16-shadow-mapping/
- Perspective Shadow Maps GPU Gems article
  https://developer.nvidia.com/gpugems/gpugems/part-ii-lighting-and-shadows/chapter-14-perspective-shadow-maps-care-and-feeding
- Perspective Shadow Maps SIGGRAPH 2002 paper
  https://www-sop.inria.fr/reves/Basilic/2002/SD02/PerspectiveShadowMaps.pdf